Lab5_Granger

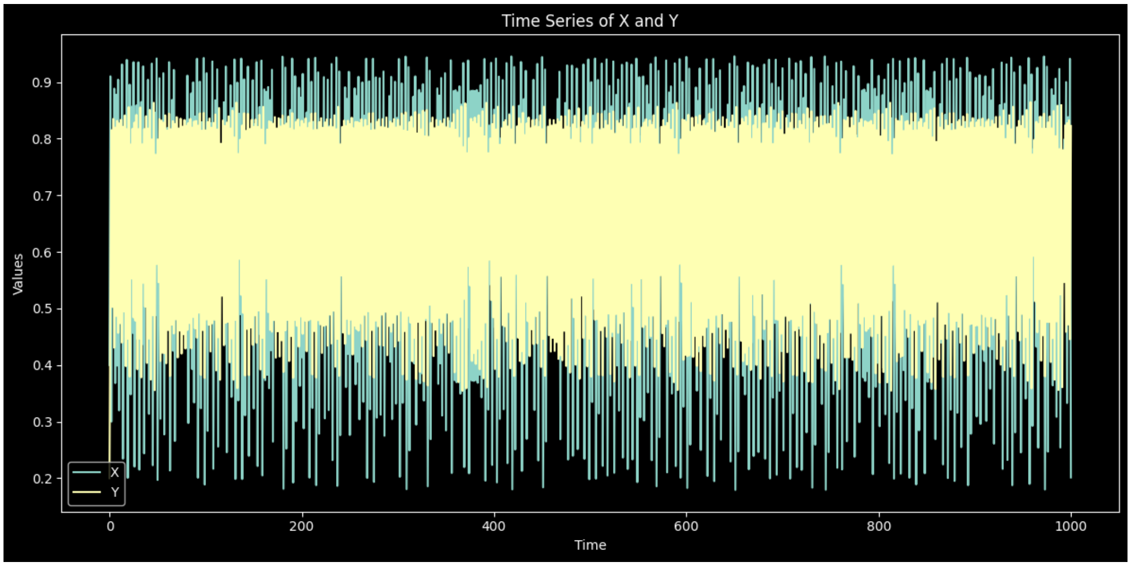
December 1, 2025

[4]:
```
#Granger causality with Chaotic series
!pip3 install statsmodels

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

'''
eyes = pd.read_csv('eyes_mean.csv', header=None)
visual = pd.read_csv('visual_raw.csv', header=None)
somatomotor = pd.read_csv('somatomotor_raw.csv', header=None)
n_subjects = eyes.shape[1]
'''
'''
subject_file = "sub-102109_TS.txt"
time_series = np.genfromtxt(subject_file, delimiter='\t')[:,1:]
'''
n_subjects = 1


def chaotic_func(A, B, r, beta):
    return A * (r - r * A - beta * B)

# params
r_x = 3.8
r_y = 3.5
B_xy = 0.02 # effect on x given y (effect of y on x)
B_yx = 0.1 # effect on y given x (effect of x on y)

X0 = 0.4 # initial val following Sugihara et al
Y0 = 0.2 # initial val following Sugihara et al
t = 1000 # time steps  #reduce to 50 to visualize them better

X = [X0]
Y = [Y0]
```

```python
for i in range(t):
    X_ = chaotic_func(X[-1], Y[-1], r_x, B_xy)
    Y_ = chaotic_func(Y[-1], X[-1], r_y, B_yx)
    X.append(X_)
    Y.append(Y_)

# Plot the results
plt.figure(figsize=(12, 6))

# Subplot for X and Y over time

plt.plot(range(t + 1), X, label="X")
plt.plot(range(t + 1), Y, label="Y")
plt.title("Time Series of X and Y")
plt.xlabel("Time")
plt.ylabel("Values")
plt.legend()


plt.tight_layout()
plt.show()


primary =  X
supplementary = Y
```

```
Requirement already satisfied: statsmodels in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(0.14.5)
Requirement already satisfied: numpy<3,>=1.22.3 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from statsmodels) (1.26.4)
Requirement already satisfied: scipy!=1.9.2,>=1.8 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from statsmodels) (1.11.4)
Requirement already satisfied: pandas!=2.1.0,>=1.4 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from statsmodels) (2.3.3)
Requirement already satisfied: patsy>=0.5.6 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from statsmodels) (1.0.2)
Requirement already satisfied: packaging>=21.3 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from statsmodels) (24.2)
Requirement already satisfied: python-dateutil>=2.8.2 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from pandas!=2.1.0,>=1.4->statsmodels) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
```

```
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from pandas!=2.1.0,>=1.4->statsmodels) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from pandas!=2.1.0,>=1.4->statsmodels) (2025.2)
Requirement already satisfied: six>=1.5 in
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages
(from python-dateutil>=2.8.2->pandas!=2.1.0,>=1.4->statsmodels) (1.17.0)
```



[5]:
```
#Do not run this for the blind experiment
#np.shape(time_series)
#primary =  time_series[:,7]
#supplementary = time_series[:,64]
```

[6]:
```
#Stationarity test (not necessary all the time)
from statsmodels.tsa.stattools import adfuller

for x in range(n_subjects):
  #Compute the stationary test, change the variable to test the other signals
  dftest = adfuller(supplementary, autolag="AIC") #.loc[:,x]

  #Print the p-values showing <0.05 if the test of stationarity is passed
  print(dftest[1])
```

```
3.051340613840355e-11
```

[8]:
```
from statsmodels.tsa.stattools import grangercausalitytests
# Perform Granger-Causality test
max_lags = 1
```

```
ROIs = ['primary','supplementary' ]# ['visual','somatomotor','eyes']#
causal_mat = np.zeros((len(ROIs), len(ROIs)))

def find_causality(max_lags,seriesto,seriesfrom):
  pvals = []
  for x in range(n_subjects):
    print(x)
    # Check whether the first variable is caused by the second one
    #df = pd.DataFrame(list(zip(seriesto.loc[:,x], seriesfrom.loc[:,x])))
    df = pd.DataFrame(list(zip(seriesto, seriesfrom)))

    # Difference to make the signal further stationary (it can also smooth␣
 ↪further the signal loosing information)
    #df = pd.DataFrame(list(zip(np.diff(visual.loc[:,x])[1:], np.diff(eyes.loc[:
 ↪,x])[1:])))
    #df = pd.DataFrame(list(zip(np.diff(seriesto)[1:], np.diff(seriesfrom)[1:
 ↪])))

    gc_res = grangercausalitytests(df, maxlag=max_lags)
    # Remove List and Dict
    gc_res_lags =  (gc_res[max_lags])
    unlisted_gc_res_lags = gc_res_lags[0]
    pval = unlisted_gc_res_lags['ssr_ftest'][1]
    pvals.append(pval)
    #print(pvals)
    avg_pval = np.mean(pvals)
  return avg_pval,pvals

# Use a for loop to iterate over the list
j_ind = 0
for ROI_i in ROIs:
  i_ind = 0
  for ROI_j in ROIs:
     avg_pval,pvals = find_causality(max_lags, locals()[ROI_i] ,locals()[ROI_j])
     causal_mat[i_ind, j_ind] = avg_pval
     i_ind = i_ind + 1
  j_ind = j_ind + 1

print(causal_mat)
causal_mat<0.05
```

```
0

Granger Causality
number of lags (no zero) 1
ssr based F test:         F=0.0000  , p=1.0000  , df_denom=998, df_num=1
ssr based chi2 test:   chi2=0.0000  , p=1.0000  , df=1
```

```
likelihood ratio test: chi2=-0.0000 , p=1.0000  , df=1
parameter F test:         F=792.6401, p=0.0000  , df_denom=998, df_num=1
0

Granger Causality
number of lags (no zero) 1
ssr based F test:         F=1.3232  , p=0.2503  , df_denom=997, df_num=1
ssr based chi2 test:   chi2=1.3272  , p=0.2493  , df=1
likelihood ratio test: chi2=1.3264  , p=0.2495  , df=1
parameter F test:         F=1.3232  , p=0.2503  , df_denom=997, df_num=1
0

Granger Causality
number of lags (no zero) 1
ssr based F test:         F=15.1316 , p=0.0001  , df_denom=997, df_num=1
ssr based chi2 test:   chi2=15.1772 , p=0.0001  , df=1
likelihood ratio test: chi2=15.0632 , p=0.0001  , df=1
parameter F test:         F=15.1316 , p=0.0001  , df_denom=997, df_num=1
0

Granger Causality
number of lags (no zero) 1
ssr based F test:         F=0.0000  , p=1.0000  , df_denom=998, df_num=1
ssr based chi2 test:   chi2=0.0000  , p=1.0000  , df=1
likelihood ratio test: chi2=-0.0000 , p=1.0000  , df=1
parameter F test:         F=13283.1809, p=0.0000  , df_denom=998, df_num=1
[[9.99999469e-01 1.06917581e-04]
 [2.50285714e-01 9.99999540e-01]]
```

```
[8]: array([[False,  True],
            [False, False]])
```